

Звіт про виконання завдання «HR-система в трьох моделях даних»

Вяльцев Олександр

1. Які бази даних ви обрали для кожної з трьох моделей і чому саме ці?

PostgreSQL (реляційна модель). Це одна з найпоширеніших реляційних СУБД з відкритим кодом. Вона повністю підтримує стандарт SQL. PostgreSQL уже був встановлений, і з ним у мене найбільше практичного досвіду, тому я міг зосередитися на проєктуванні схеми та запитах, а не на розбиранні з інструментом.

MongoDB (документна модель). Це найпопулярніша документна СУБД. Резюме в ній можна зберігати одним документом: вкладені масиви хобі й досвіду роботи, місто як поле. Тому вона добре показує сильні й слабкі сторони документного підходу. Для унікальних значень є `distinct`, для складніших вибірок — `Aggregation Pipeline`. З MongoDB я теж уже працював, хоч і менше, ніж із PostgreSQL.

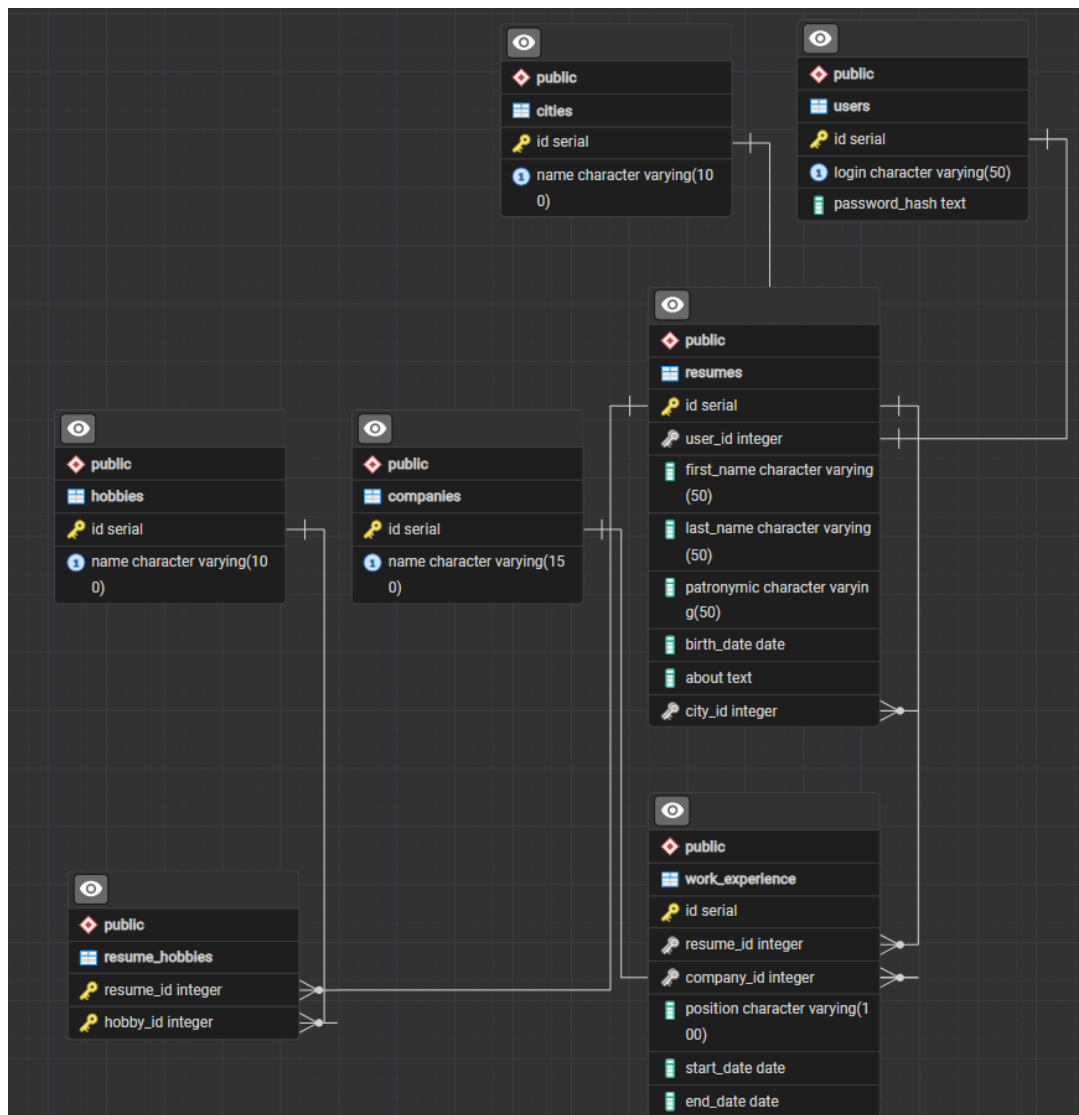
Neo4j (графова модель). Досвіду з графовими базами в мене не було. Тому я порівняв кілька варіантів: Neo4j, Memgraph, ArangoDB, Apache AGE, JanusGraph/Amazon Neptune. Обрав Neo4j з таких причин:

- це найпоширеніша графова СУБД, тому для неї найбільше документації та прикладів;
- мова запитів Cypher описує зв'язки шаблонами і її легко читати; на основі Cypher створено міжнародний стандарт ISO GQL;
- це «чиста» графова база: на відміну від мультимодельної ArangoDB чи розширення Apache AGE для PostgreSQL, вона не змішує модель з іншими, тож порівняння трьох моделей виходить чесним;
- безкоштовна Community Edition легко встановлюється, а вбудований Neo4j Browser показує граф наочно.

Отже, у кожній моделі взято типового, найпоширенішого представника. Так отримані висновки можна узагальнювати на всю модель, а не на особливості конкретного продукту.

2. Як саме ви реалізували зв'язок один-до-багатьох і багато-до-багатьох у кожній з трьох моделей?

PostgreSQL. Сім таблиць. Користувач і резюме пов'язані один-до-одного через `resumes.user_id` UNIQUE. Місто винесене в окремий довідник `cities`, і кожне резюме посилається на нього зовнішнім ключем. Це зв'язок один-до-багатьох: в одному місті живе багато здобувачів. Багато-до-багатьох реалізовано двічі через проміжні таблиці. `resume_hobbies` пов'язує резюме з довідником `hobbies`. `work_experience` пов'язує резюме з довідником `companies` і додатково зберігає посаду та період роботи.



MongoDB. Одна колекція users. Резюме вкладене прямо в документ користувача, бо вони завжди читаються разом. Окремих довідників немає, тому зв'язки неявні. Один-до-багатьох: місто зберігається рядком resume.city, і багато документів мають однакове значення. Багато-до-багатьох: масив рядків hobbies і масив вкладених об'єктів experience, де той самий заклад зустрічається в різних документах.

```

{
  "login": "oleksandr_kovalenko",
  "password_hash": "...",
  "resume": {
    "first_name": "Олександр", "last_name": "Коваленко", "patronymic": "Миколайович",
    "birth_date": ISODate("1987-03-24"), "about": "...",
    "city": "Дніпро",
    "hobbies": ["малювання", "подорожі"],
    "experience": [
      { "company": "EPAM", "position": "UI/UX Designer",
        "start_date": ISODate("2009-03-01"), "end_date": ISODate("2012-07-01") }
    ]
  }
}

```

Цілісність даних база тут не контролює. Якби в одному документі було «EPAM», а в іншому «Ерам», MongoDB вважала б їх різними закладами.

Neo4j. Сутності — вузли, зв'язки — ребра. Один-до-багатьох: від кожного резюме йде одне ребро LIVES_IN до вузла міста, а до одного міста ведуть ребра від багатьох резюме. Багато-до-багатьох: ребра HAS_HOBBY до вузлів хобі та WORKED_AT до вузлів закладів. Посада й період зберігаються як властивості самого ребра WORKED_AT, тому проміжна сутність не потрібна. Унікальність логіна, міст, хобі й закладів забезпечують обмеження UNIQUE.

```
(:User {login, password_hash})-[:HAS_RESUME]->(:Resume {first_name, last_name, patronymic, birth_date, about})
(:Resume)-[:LIVES_IN]->(:City {name})
(:Resume)-[:HAS_HOBBY]->(:Hobby {name})
(:Resume)-[:WORKED_AT {position, start_date, end_date}]->(:Company {name})
```

У графі немає формальної різниці між один-до-багатьох і багато-до-багатьох: це просто ребра. Кількість зв'язків контролює застосунок, а не схема.

3. Для кожного із 5 запитів: у якій моделі запит вийшов найпростішим для написання, а в якій — найскладнішим?

Запит	Найпростіше	Найскладніше
1. Резюме користувача	MongoDB	PostgreSQL
2. Унікальні хобі	MongoDB	PostgreSQL ≈ Neo4j
3. Унікальні міста	MongoDB	PostgreSQL ≈ Neo4j
4. Хобі мешканців міста	MongoDB ≈ Neo4j	PostgreSQL
5. Спільний заклад	Neo4j	MongoDB

Запит 1. У MongoDB резюме вже лежить одним документом, тож вистачило одного find. У PostgreSQL дані розкидані по семи таблицях. Щоб зібрати одне резюме, довелося писати підзапити з array_agg і json_agg. Звичайний JOIN хобі й досвіду дав би декартів добуток: 2 хобі × 3 місця роботи = 6 рядків замість одного.

Запити 2 і 3. У MongoDB це один виклик distinct, причому він сам розгортає масив хобі. У PostgreSQL і Neo4j запити теж короткі, але потребують JOIN або шаблону зв'язку, щоб брати лише ті хобі й міста, які справді зустрічаються в резюме.

Запит 4. У MongoDB це той самий distinct, тільки з фільтром. У Neo4j теж дуже просто: місто можна задати прямо в шаблоні зв'язку (с:City {name: "Київ"}). Фактично це аналог JOIN разом із WHERE у PostgreSQL, але записаний як одна умова. У PostgreSQL знадобилося три JOIN, щоб пройти шлях хобі → резюме → місто.

Запит 5. Тут різниця найбільша. Детально про неї в питанні 4.

Загалом MongoDB виявилася найзручнішою для простих запитів 1–4, бо дані вже лежать у потрібному вигляді. Але щойно запит виходить за межі одного документа, вона стає найскладнішою.

4. Запит №5 зазвичай найбільше відрізняється складністю між моделями. Опишіть, з якими труднощами ви зіткнулися при його реалізації в кожній з трьох баз — і зробіть висновок, чому так вийшло.

Neo4j — найпростіше. Запит фактично описує зв'язок словами: користувач → резюме → заклад. Далі збираємо людей кожного закладу через collect і лишаємо заклади, де їх більше одного. Новим для мене було лише ключове слово WITH. Воно передає результат агрегації на наступний крок і за змістом відповідає HAVING в SQL.

PostgreSQL — звичний, але громіздкий запит. Логіка стандартна: GROUP BY за закладом, HAVING count(...) > 1 і array_agg для списку людей. Але щоб пройти шлях заклад → досвід роботи → резюме → користувач, знадобилося чотири таблиці й три JOIN. Окремо довелося стежити за DISTINCT всередині агрегації, щоб людина, яка працювала в закладі двічі, не потрапила в список двічі.

MongoDB — найважче. У документній моделі заклад не є окремою сутністю: це просто рядок усередині вкладеного масиву в кожному документі. Щоб згрупувати людей за закладом, довелося «вивернути» дані навиворіт конвеєром з п'яти етапів:

- \$unwind розгортає масив досвіду;
- \$group групує за назвою закладу;
- \$addToSet збирає унікальних людей;
- \$match з \$expr і \$size фільтрує заклади, бо порівняти довжину обчисленого масиву звичайним фільтром не можна;
- \$project перейменовує поле.

Синтаксис із \$-операторами й глибокою вкладеністю дужок важко читати й налагоджувати. Крім того, результат залежить від того, що назви закладів записані однаково в усіх документах, а база цього не гарантує.

Спільна помилка в усіх трьох базах. Спочатку я ідентифікував людей за ім'ям і прізвищем. Перевірка показала, що в даних є повні тезки: дві «Наталії Гончаренко», три «Сергії Шевченки». Тому DISTINCT / \$addToSet / collect(DISTINCT) склеювали різних людей в одну. Наприклад, у MacPaw запит показував 7 осіб замість 8. Виправлення однакове в усіх моделях: розрізняти людей за унікальним логіном.

Чому так вийшло. Запит №5 — це запит про зв'язки між сутностями через спільну третю сутність: резюме → заклад ← резюме. Кожна модель обробляє зв'язки по-своєму:

- у графовій моделі зв'язки зберігаються явно як ребра, тож запит — це просто шаблон шляху;
- у реляційній моделі зв'язки відновлюються під час запиту через JOIN за зовнішніми ключами;
- документна модель оптимізована під читання одного документа цілком (тому запит №1 у ній найпростіший); зв'язки між документами в ній не передбачені, і їх доводиться будувати вручну, перебудовуючи дані в конвеєрі.

Отже, складність запиту залежить від того, наскільки він відходить від «одиниці» моделі: для документної це один документ, для графової — зв'язок.

5. Якби ви створювали цю HR-систему в реальному проєкті, яку модель (чи комбінацію моделей) ви б обрали і чому?

MongoDB. З нею найприємніше було писати прості запити 1–4: резюме лежить одним документом і читається одним викликом. Ще одна перевага — простота структури. Для всієї системи вистачило однієї колекції, тоді як у PostgreSQL довелося спроектувати сім таблиць із зовнішніми ключами та проміжними таблицями. Але на запиті №5 стало видно межі цієї моделі. Щойно треба шукати зв'язки між різними резюме, запити стають громіздкими й незручними. Крім того, база не контролює цілісність даних: «EPAM» і «Ерап» для неї різні заклади, і це помітно впливає на результати запитів на кшталт №5.

PostgreSQL. У ньому всі п'ять запитів вийшли без особливих труднощів. Вони трохи довші, але стандартні й передбачувані. Довідники й зовнішні ключі гарантують цілісність даних, UNIQUE гарантує унікальність логіна, а транзакції захищають облікові записи і паролі. До того ж з PostgreSQL у мене найбільше досвіду. Зручність документної моделі теж можна частково отримати в Postgres: менш структуровані частини резюме можна зберігати в полі типу JSONB.

Neo4j. Він найбільше мене вразив. Запит №5, найскладніший у двох інших базах, у ньому вийшов найкоротшим і найзрозумілішим. Його перевага над PostgreSQL з'являється тоді, коли запити йдуть через кілька зв'язків. Наприклад, у HR-системі можуть знадобитися:

- «колеги колег» кандидата;
- рекомендації кандидатів через спільних роботодавців;
- найкоротший ланцюжок знайомств між двома людьми.

У SQL кожен такий крок — це ще один JOIN або рекурсивний запит, і складність швидко росте. У Cypher достатньо змінити довжину шляху в шаблоні, наприклад `-[:WORKED_AT*1..3]-`. Але для основних операцій HR-системи (реєстрація, зберігання й редагування резюме, облікові записи) реляційна база надійніша, і з нею простіше працювати.

Висновок. Вибір моделі залежить від того, які запити система виконуватиме найчастіше.

- Якщо систему потрібно лише для зберігання й перегляду резюме, а точність назв компаній і міст не критична, цілком підійде MongoDB: одна колекція, мінімум проектування й найпростіші запити для типових операцій.
- Якщо потрібна аналітика по резюме (хто де працював, спільні заклади, вибірки за містами) і важлива цілісність даних, я б обрав PostgreSQL як основну базу.
- Якщо з'являться функції пошуку зв'язків між людьми (рекомендації, мережа колег, ланцюжки знайомств), до PostgreSQL варто додати Neo4j саме для цих запитів.

Для HR-системи з цього завдання, де запит №5 показовий, з власного досвіду я б зупинився на PostgreSQL.